

Simple E-Commerce Website



(relevant image once front end is developed)

Authors

Isak Hansson | sikhan-3@student.ltu.se

Ask Frössling | askfro-3@student.ltu.se

Abdulsalam Alkalash | abdala-3@student.ltu.se



March 15, 2026

Executive Summary

This project aims to design and implement a small-scale e-commerce web application selling mugs with a strong focus on relational database as a part of the D0018E course. The system will allow users to browse products, place orders, and interact with stored data through a web-based interface. The solution is intended to be deployable on a cloud-based virtual machine and to serve as a portfolio-ready demonstration of practical database and web development skills.

The project follows an agile and incremental development approach inspired by Scrum, with development divided into short sprints and continuous documentation.

A relational SQL database is used as the core of the system to store and manage data related to products, customers, and orders. The back end is implemented using Python and Flask, while a lightweight frontend set up with Vue provides basic user interaction through a web browser. Furthermore the system is designed to be deployable on a cloud platform such as Amazon Web Services (AWS).

Github repository link: https://github.com/KasiisaK/D0018E_Project

Contents

1	Project Overview & Scope	iii
2	Requirements & Backlog	iv
2.1	User stories	iv
2.2	Product backlog	v
3	System Architecture	vi
3.1	Technology choices & motivation	vi
3.2	Frontend architecture	vi
3.2.1	Index.html and groundwork	vi
3.2.2	Backend communication with Axios	vii
3.2.3	State management with Pinia	vii
3.3	Backend architecture & database	viii
3.3.1	Database schema	viii
3.3.2	Backend API with Flask	ix
3.3.3	JWT decorator	ix
3.4	Deployment environment	x
4	Testing	xi
4.1	Database testing	xi
4.2	API testing	xi
4.3	First GUI test	xi
4.4	Final testing	xii
5	System limitations and improvements	xiii
5.1	Schema limitations	xiii
5.1.1	Cart limitations	xiii
5.1.2	Image limitations	xiii
5.1.3	Deleting products stored in an order	xiii
5.1.4	Any user reviewing	xiii

1 Project Overview & Scope

The system is designed as a web-based e-commerce application consisting of three main components: a frontend client using Vue, a backend application server using Python + Flask[5] + MySQL Connector[3] + Axios[2], and a relational database MySQL[10]. Users interact with the system through a standard web browser, while all persistent data is stored and managed in the database.

The frontend provides a minimalistic user interface focused on core functionality such as viewing available products and initiating orders. As the course emphasizes database design rather than graphical design, the frontend is intentionally kept simple.

The backend application is responsible for handling business logic, processing user requests, and performing database operations. It exposes a set of endpoints that the frontend can use to retrieve and modify data. The database layer stores information about products, customers, and orders (shopping carts).

The system is intended to be deployed on a virtual machine in a cloud environment, such as Amazon Web Services (AWS)[1]. This allows the application to be accessed remotely and demonstrates how a database-backed web application can be hosted on an external server. In Sprint 0, the implementation effort is limited to architectural planning, technology selection, while functional features will be developed incrementally in later sprints.

Furthermore the system will not have any tractional capabilities; this project is mainly focused on the backend database design. The entire system can be viewed as a prototype that only encompass a light front end with a bare bones backend.

2 Requirements & Backlog

2.1 User stories

The following user stories describe the core functionality required for the minium viable product. The stories are defined from the perspective of two actors: a customer and an administrator.

Customer

- As a customer, I want to view a list of products so that I can see what items are offered in the store.
- As a customer, I want to add a product to a shopping cart so that I can prepare an order.
- As a customer, I want to remove items from the cart and change the amount purchased.
- As a customer, I want to be able to view my final order.

Administrator

- As an administrator, I want to add new products to the database so that they are available for customers.
- As an administrator, I want to update product information and price.

2.2 Product backlog

The product backlog is derived from the user stories and should reflect how the system should behave based on actors needs.

ID	Description	Sprints
01	Select system architecture and technology stack	0
02	Write documentation (this report)	all
03	Set up GitHub repository and coding environment	1
04	Make user stories	1
05	Making a schema for the MySQL database	1
06	Make a web front end prototype	2
07	Make API endpoints	2
08	Test API implementation with Postman	2
09	Have a first prototype where (a4.3) user is able to brows products	2
10	Have a first prototype where (a) user is able to add a product to the cart	2
11	Have a first prototype where (a) user is able to view their cart	2
12	Have a first prototype where (a) user is able change the quantity of a product in the cart	2
13	Switch to pinia as data mediator	3
14	Being able to sign up	3
15	Being able to log in and be given a session token	3
16	Expand database for more attributes (rating, reviews, etc)	3
17	A user is view a detailed product description	3
18	A user final order is placed upon check out	3
19	Fix AWS environment to host the website	3
20	A user is able to make review on a product	4
21	Make so admins can edit products (name, price, description)	4
22	Make more products and better descriptions for final release	4

Table 1: Sprint Backlog

3 System Architecture

The system is designed using a classic three-tier architecture consisting of a frontend client, a backend application server, and a relational database. Each layer has a clearly defined responsibility, which improves maintainability and interoperability.

The frontend is implemented using Vue and runs in the user's web browser. It is responsible for presenting data to the user and collecting user input, such as browsing products or modifying the shopping cart. The frontend does not communicate directly with the database; instead, all interactions are performed through the backend via HTTP requests using Axiom as a mediator.

The backend is implemented in Python using the Flask framework and acts as the central coordinator of the system. It exposes a set of REST-style API endpoints that the frontend can call to retrieve or modify data. The backend contains the application logic such as: including validation of user input, handling cart operations and more.

Persistent data storage is handled by a MySQL relational database. Communication between the backend and the database is performed using the MySQL Connector for Python. This layer is responsible for storing and retrieving structured data related to users, products, and cart items, while ensuring data integrity through constraints and relational links.

The system is intended to be deployed in a cloud-based environment, such as Amazon Web Services (AWS), where the backend and database can run on a virtual machine. This deployment approach allows the application to be accessed remotely and mirrors real-world production environments. During development, all components can also be run locally to simplify testing.

3.1 Technology choices & motivation

In designing the system, our primary goal was to keep the development process as simple and accessible as possible while still using modern, industry-relevant technologies. We deliberately chose tools that minimize configuration overhead and unnecessary boilerplate code, so that more time could be spent on implementing functionality rather than managing a complex infrastructure.

For the backend, we selected Python together with the Flask. Where python is widely known for its simple syntax. Flask was chosen because it is a lightweight web framework that provides only the essential components needed to build a web API. This minimalism allowed us to implement REST endpoints quickly and iterate design prototypes faster.

For the frontend, we chose Vue, which offers a component-based architecture similar to React (React) but is generally considered easier to learn albeit at the cost of some features. Its clear separation between template, logic, and styling helped us build user interface components in a structured way. Vue also requires less initial configuration than many alternatives, enabling us to focus on implementing user interactions rather than tooling.

Overall, the technology stack of Python + Flask for the backend and Vue for the frontend reflects a deliberate trade-off: prioritizing simplicity, readability, and rapid development over complexity and heavy abstraction. This combination allowed us to build a functional full-stack application with a relatively low barrier to entry, making the project easier to understand for first timers like ourselves.

Moreover some smaller libraries such as mysql-connector-python and Axios were chosen for the same: easy to read and understand syntax, with some nice to have features for example Axiom automatically converting JASON objects.

3.2 Frontend architecture

Here we will go over parts making up our general frontend architecture. This will cover how we, through the use of VUE[13]. Vue is a lightweight modular component based framework that makes it easy to create a modern reactive frontend.

3.2.1 Index.html and groundwork

Since the Vue framework builds on its own file-format, our initial and only `.html` file is kept general and simple, only containing a header to preload fonts. Then a body referencing our `main.js` containing the `routes` linking different views together, and our `root` element to mount the Vue app. Once the groundwork is laid out Vue files can be created, one for every major tab of the website, these include the:

- **Home** - Where some of the best selling products will be displayed .
- **Products** - Where the full website catalogue of products can be viewed.
- **About & Contact** - Where website information is stored.

- **Cart** - Where users can view their shopping carts.
- **Sign up & in** - Where new users can sign up or returning users can sign in.

3.2.2 Backend communication with Axios

For the front end to communicate with the REST APIs Axios[2] was used. Where Axios is a popular, open-source JavaScript library used to send asynchronous HTTP requests to REST endpoints. For ease of testing and deployment, an `index.js` file handles the base URL, making it easier to swap out the base URL for local testing. This is done with the `axios.create([config])` which creates a custom config where you include the aforementioned `baseURL`, but also things like `timeout` which makes sure the server does not hang itself and times out after a while instead. Furthermore the `index.js` also handles token authentication in a centralized manner, with the `interceptors.request` that intercepts every network request; which is used to automatically places the JSON Web Token (JWT)3.3.3 on every network request (if applicable).

3.2.3 State management with Pinia

State management in the frontend application is handled using Pinia[9], which can be thought of as a middle man between the backend and frontend. Pinia provides a centralized storage that reacts in real-time as opposed to having to make queries to update the state of the website. This approach allows a shared state to be accessed across multiple components.

In the system, Pinia stores are used to manage both authentication and shopping cart functionality. The authentication store maintains the current user object and JWT token3.3.3, as well as computed properties such as whether the user is authenticated. The cart store manages cart items, total quantities, and total price, while also encapsulating all API communication related to cart operations.

Using Pinia improves separation of concerns by keeping business logic and state handling outside of individual components. Instead of duplicating logic across views, components simply import the relevant store and interact with that instead. This centralized approach results in higher level of code abstraction aiding in reusability and maintainability.

3.3 Backend architecture & database

This section describes how the backend is set up to handle a typical e-commerce website requests to a MySQL database using a RESTful API set up in Flask.

3.3.1 Database schema

The database schema took inspiration from[7] and applied it to the systems own needs; resulting in the following Entity-Relationship Diagram (ERD).

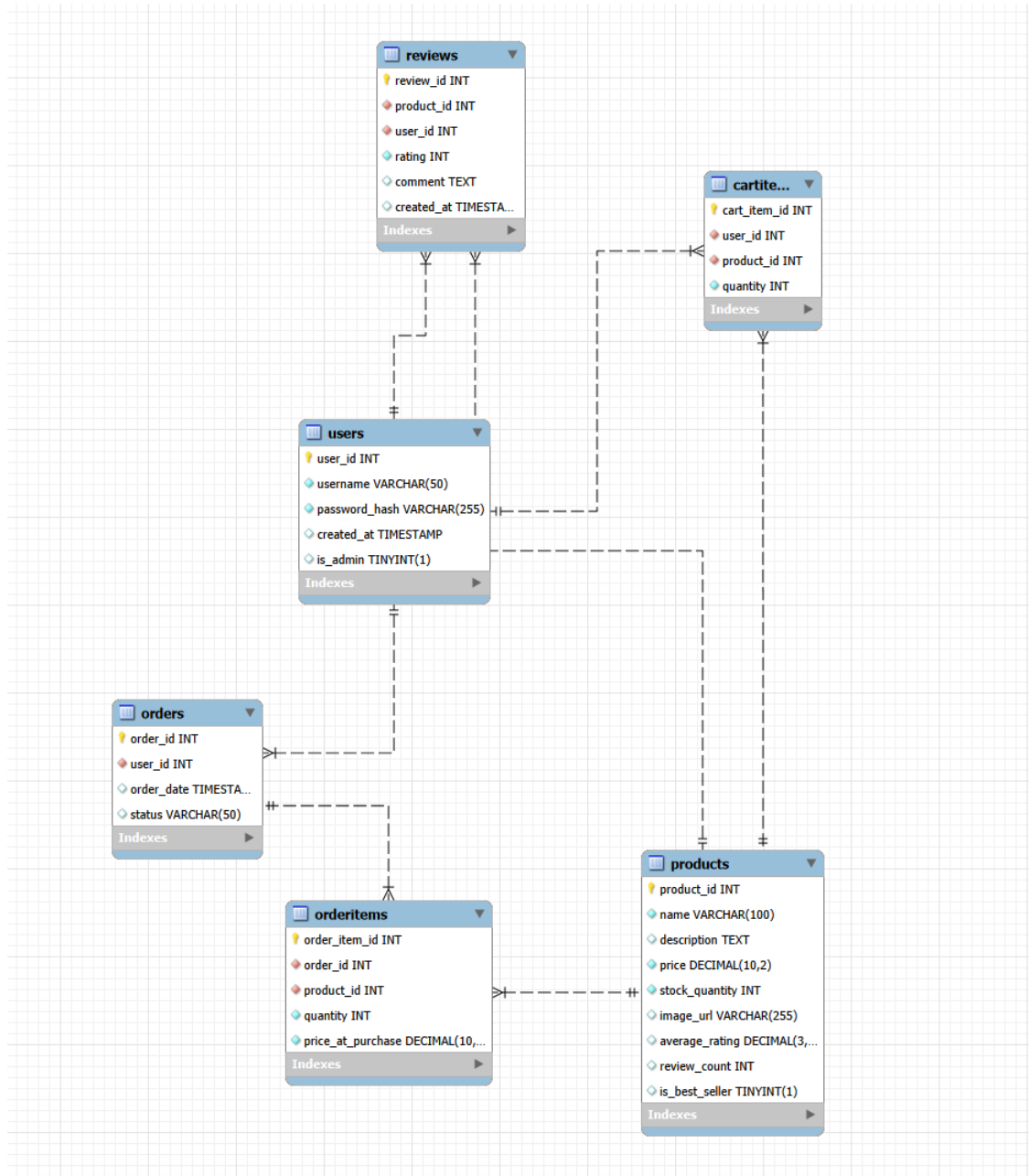


Figure 1: Database schema

The *Users* table represents registered users of the system. Each user is uniquely identified by the attribute **user_id**, which serves as the primary key. The **username** attribute is constrained to be unique to prevent duplicate user accounts, while **password_hash** stores a securely hashed version of the user's password. The **created_at** attribute records when the user account was created (not used currently, mainly for future expansion).

The *Products* table contains information about items available for purchase. Each product is uniquely identified by the primary key **product_id**. Additional attributes include the product name, description, price,

available stock quantity, and an optional image URL. Storing stock quantity in the database allows the system to track inventory levels and prevent overselling.

The *CartItems* table represents the contents of a user's shopping cart, where each cart item is stored as a separate row. Each cart item references exactly one user via the **Users** *user_id* and one **Product(s)** through the foreign key *product_id*. The foreign keys are also set to be unique stopping duplicate items in the **Users** cart. Furthermore limitations about this approach are discuss here5.1.1.

For the *cartItems* table three constraints[6] where used to simplify later implementations. The first two constraints checks if foreign keys for *user_id* and *product_id* already exist in there respective tables. The last constraint ensures that no duplicates of items can occur in a users cart.

The *Orders* table represents completed purchases made by users and provides persistent storage of order history. Each order is uniquely identified by the primary key *order_id* and is associated with exactly one user through the foreign key *user_id*. This design allows a single user to place multiple orders over time, establishing a one-to-many relationship between *Users* and *Orders*. Attributes also include a *status* for what stage the order is in.

The *OrderItems* table represents the individual products contained within each order. Each row corresponds to one product included in a specific order and is referenced via the foreign key *order_id*. Then the *product_id* is likewise a foreign key referencing which product is in the order. Notably the attributes for identifying the quantity and price of an order are slighting different with the price being *price_at_purchase*; this makes sure to save the price of the product when the user bought the product.

Together, the *Orders* and *OrderItems* tables model the checkout process of the system, where items from a user's cart are converted into a persistent order record. This design supports multiple orders per user and multiple products per order while maintaining data integrity through relational constraints.

The *Reviews* table stores the a **Users** review on a product. The user selects a *rating* from 1/5 and has the option to write a accompanying *comment*, for future expansion refer to5.1.4

3.3.2 Backend API with Flask

For initial setup and understanding the following tutorial was used[12].

The system uses Flask to manage API requests in the form of a RESTful API. To connect to the running MySQL database, the MySQL Connector for Python (*mysql-connector-python*) is used. In this case, connecting to the local database is accomplished by providing the correct values for the fields *host*, *user*, *password*, and *database*. Upon successful connection to the database, a cursor can be created, which enables the execution of SQL queries.

Data transmitted through the API is formatted using JSON[4], as Flask provides native support for this format. JSON is widely used for web communication and therefore ensures compatibility with various frontend implementations if desired.

Each endpoint corresponds to a specific URL path and HTTP method, allowing the frontend to retrieve or modify data in the database. When a request is received, Flask executes the associated Python function defined for that route.

Database operations are performed using the MySQL Connector for Python. After establishing a connection, a cursor object is created, which allows SQL queries to be executed. Parameterized queries are used, where placeholders *%s* represent values supplied separately from the SQL string. This approach prevents SQL injection and ensures safe handling of user inputs.

Images used for displaying a product are stored in a Flask designated folder */static* where any image can be accessed by *http://<server>/static/<filename>* making it easy for the front end to load the image later by referencing that specific URL. Limitations of this approach is discuss here5.1.2

3.3.3 JWT decorator

To secure protected endpoints in the backend, a JSON Web Token (JWT)[8] authentication mechanism is implemented through a custom decorator. This decorator, named *token_required*, ensures that only authenticated users can access specific routes, such as those related to cart management and order creation.

The decorator operates by intercepting incoming HTTP requests before the associated route function is executed. It checks for the presence of an **Authorization** header and extracts the token from the **Bearer** scheme. If no token is provided, the request is immediately rejected with a **401 Unauthorized** response.

If a token is present, it is decoded using the server's secret key and the HS256 signing algorithm. During decoding, the token's signature and expiration time are verified. If the token is invalid, malformed, or expired, access is denied and an appropriate error message is returned. If the token is successfully validated, the user's

identifier embedded within the token payload is attached to the request object (e.g., `request.user_id`). This allows subsequent route logic to securely determine the identity of the authenticated user.

The use of a decorator provides a clean and reusable solution for authentication enforcement. By simply annotating a route with `@token_required`, the same authentication logic can be consistently applied across multiple endpoints. This centralized approach reduces code duplication and increases the maintainability of the system.

3.4 Deployment environment

The system is deployed using Amazon Web Services (AWS), specifically Amazon RDS for database management and AWS Elastic Beanstalk for backend deployment. The relational database is hosted on Amazon RDS running MySQL, which provides a managed database environment including automated backups, scaling capabilities with more expensive paid plans. Using RDS removes the need to manually configure or maintain the database server, allowing the focus to remain on application development.

The backend application, built using Flask, is deployed via AWS Elastic Beanstalk. Elastic Beanstalk simplifies deployment by automatically provisioning the necessary infrastructure, load balancing, and environment configuration. It allows new application versions to be uploaded and deployed with minimal manual configuration.

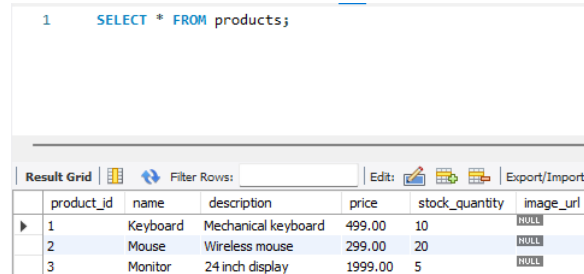
Database connectivity is handled using the `mysql-connector-python` library. To ensure security and flexibility, sensitive configuration parameters such as database host, username, password, and database name are not hardcoded in the source code. Instead, they are retrieved using `os.getenv()`, which reads environment variables at runtime. In the production environment, these variables are configured directly within the Elastic Beanstalk environment settings. For local development and testing, a `.env` file can be used to define the same variables, allowing the application to run in different environments without modifying the source code. This separation of configuration from code improves security of the system.

4 Testing

4.1 Database testing

The early stage testing reflect the user stories, meaning that the testing is done to ensure that the user stories can be realised.

The first test was to see if the products could be queried



	product_id	name	description	price	stock_quantity	image_url
▶	1	Keyboard	Mechanical keyboard	499.00	10	NULL
	2	Mouse	Wireless mouse	299.00	20	NULL
	3	Monitor	24 inch display	1999.00	5	NULL

Figure 2: Test case 1

The next test was to make sure a **user** could add items to their cart `INSERT INTO cartitems (user_id, product_id, quantity) VALUES (1, 2, 1);` showing

	cart_item_id	user_id	product_id	quantity
▶	11	1	2	1

Figure 3: Test case 2

Then testing if a product could be removed with `DELETE FROM cartitems WHERE user_id=1 and product_id=2;`

	cart_item_id	user_id	product_id	quantity
*	NULL	NULL	NULL	NULL

Figure 4: Test case 3

Showing that the item that was just added to the **user 1's** cart was removed.

After inserting test data in the *orderitems* table and quiering a *users* order

order_item_id	order_id	product_id	quantity	price_at_purchase
1	1	1	1	499.00
2	1	2	2	299.00

Figure 5: Test case 4

Then finally testing in product(s) can be inserted by a admin

product_id	name	description	price	stock_quantity	image_url
1	Keyboard	Mechanical keyboard	499.00	10	NULL
2	Mouse	Wireless mouse	299.00	20	NULL
3	Monitor	24 inch display	1999.00	5	NULL

Figure 6: Test case 5

4.2 API testing

Testing the API was done with Postman[11] where all necessary data paths can be tested in advance before integration with the frontend; Where API paths relevant to the user stories 2.1 where tested such as querying a users cart.

This also served as a rudimentary way of adding products to the **products** database. Simply by imputing the data in the **body** field configured in the JSON format with necessary key/value pairs.

4.3 First GUI test

After sprint one we could test some basic front back-end interactions, albeit limited in the sense that the **user** with `user_id = 1` was hardcoded as users where not able to be distinguished otherwise, since user sign up and log in was not implemented yet. Regardless every interaction with the GUI worked as intended.

4.4 Final testing

Similarly to the testing done after sprint 1 the assessment of the system was done manually, clicking on every button, filling in every field, testing edge cases, etc. For large scale enterprise deployment this should be done automated testing after git a commit, but since this is a smaller system this manual testing is sufficient.

5 System limitations and improvements

This section is to discuss the limitations of the design choices made during development of the e-commerce website.

5.1 Schema limitations

5.1.1 Cart limitations

The database architecture does not contain a table for a *Cart* meaning a *Users* would not be able to have more than one cart at a time, not one to many. We don't see this as a great problem since it's firstly rarely used, and secondly it would entail a higher level of complexity with the requirement of another table. Using a single table for the items in a cart means the query would have to parse the cart based on the `user_id` as such: `SELECT * FROM CartItems WHERE user_id = x`; where x is the *Users* `user_id`. In short making the query the cart.

5.1.2 Image limitations

Further, there are some limitations regarding the images used to display the different products, in that only a single image can be displayed per product. This is due to the foundational principle of a relational database being that all values are atomic, meaning that saving a list of images for a single product is not possible within a single field. Therefore, the system would require a dedicated table containing image entries with a foreign key referencing the corresponding product in order to support multiple images per product. The system would therefore need to be expanded to accommodate this in the future if such functionality is desired.

5.1.3 Deleting products stored in an order

Since the both the `order_items` and the `cart_items` tables have foreign referencing a product simply deleting a product would not work. One alternative could have been the use of `CASCADE` to delete all references to the product, but since this would also delete a users purchased product in the `order_items` table that would not have been an option. Rather the simple solution would have been to include a `is_visible` flag in the `products` table. This would have made it so a product could have been delisted instead of outright deleted, keeping the reference to other tables whilst hiding the product on the website.

5.1.4 Any user reviewing

Currently there is no check done in the front end to know whether or not a `user` has made a purchase of the product previously. In future expansions of the system there should be a check to see if a `user` is eligible to make the comment i.e. has purchases the product previously.

References

- [1] AWS. URL: <https://aws.amazon.com/>.
- [2] Axios. *Promise based HTTP client for the browser and node.js*. URL: <https://axios-http.com/>.
- [3] chuck.bell. *MySQL Connector/Python*. URL: <https://pypi.org/project/mysql-connector-python/>.
- [4] ECMA. *Introducing JSON*. URL: <https://www.json.org/json-en.html/>.
- [5] Flask. URL: <https://flask.palletsprojects.com/en/stable/>.
- [6] Geek For Geeks. *SQL / Constraints*. URL: <https://www.geeksforgeeks.org/sql/sql-constraints/>.
- [7] Biswanath Giri. *Designing the database schema for a new e-commerce platform and considering factors like normalization, denormalization, indexing, and performance optimization to ensure efficient storage and retrieval of data*. URL: <https://bgiri-gcloud.medium.com/designing-the-database-schema-for-a-new-e-commerce-platform-andconsiderinfactorslike-ec28d4fb81db>.
- [8] *Introduction to JSON Web Tokens*. URL: <https://www.jwt.io/introduction#when-to-use-json-web-tokens/>.
- [9] Eduardo San Martin Morote. *PiniaThe intuitive store for Vue.js*. URL: <https://pinia.vuejs.org/>.
- [10] *MySQL*. URL: <https://www.mysql.com/>.
- [11] Inc Postman. *Postman API Platform*. URL: <https://www.postman.com/product/>.
- [12] United Top Tech. *How to connect MySQL Database with Flask Tutorial*. URL: <https://www.youtube.com/watch?v=14HTiBQQ9M/>.
- [13] Evan You. *The Progressive JavaScript Framework*. URL: <https://vuejs.org/>.